Software Security — Lab 0 Cryptography Review

Applied Cryptography in Java

Development environment

You may use any Java-compatible development environment:

- IntelliJ IDEA;
- Eclipse;
- Apache NetBeans;
- Visual Studio Code with Java extensions;
- another Java IDE or editor;
- command-line development.

The development environment is not part of the assessment. The required result is a working Java 21 Maven project with meaningful security tests and a technical report.

You need:

- JDK 21;
- Maven, Maven Wrapper, or Maven support provided by your IDE;
- JUnit 5 through the supplied project;
- a terminal or IDE terminal.

Check Java:

```
java -version
javac -version
```

Use one of the following commands to run the project:

```
mvn test
```

Windows PowerShell with Maven Wrapper:

```
.\mvnw.cmd test
```

Linux/macOS with Maven Wrapper:

```
./mvnw test
```

If you use an IDE, import the directory containing `pom.xml` as a Maven project and run the Maven `test` lifecycle or the JUnit tests from the IDE.

1. Overview

The purpose of this lab is to study the use of cryptography in a software application. You will use Java cryptographic APIs to compute message digests, authenticate messages, generate secure random values, store passwords, and encrypt sensitive data with AES-GCM.

The lab focuses on common implementation mistakes. A cryptographic algorithm may be secure, but its use in software may still be unsafe if the developer:

- selects an inappropriate primitive;
- hardcodes a cryptographic key;
- uses an unsuitable random-number generator;
- reuses an IV or nonce;
- fails to authenticate ciphertext;
- confuses encoding, hashing, and encryption;
- compares sensitive values incorrectly.

The lab does not require mathematical proofs. Your task is to observe program behavior, interpret results, identify security properties, and correct the implementation.

Topics

- confidentiality, integrity, and authenticity;
- cryptographic hash functions;
- HMAC;
- secure random-number generation;
- password hashing;

- symmetric encryption;
- AES-GCM;
- IVs, nonces, authentication tags, and AAD;
- common cryptographic implementation errors;
- Java cryptographic APIs;
- JUnit security testing.

Preparatory reading

Review the following concepts before starting:

- confidentiality, integrity, and authenticity;
- cryptographic hash functions;
- HMAC;
- symmetric encryption;
- block-cipher modes;
- IVs and nonces;
- authenticated encryption;
- password hashing.

2. Project

Use the supplied `lab0-student` project:

```
lab0-student/
├── pom.xml
├── README.md
├── src/
│   ├── main/java/dz/univ/usp/
│   │   ├── VulnerableCrypto.java
│   │   ├── PasswordService.java
│   │   ├── HmacService.java
│   │   ├── TokenService.java
│   │   ├── EncryptedMessage.java
│   │   └── AesGcmService.java
│   └── test/java/dz/univ/usp/
```

```
│   └── CryptoLabTest.java
└── data/
    └── grades.json
```

Keep the package name and the public method signatures unchanged so that the automated tests can run correctly.

The initial project contains incomplete methods and a deliberately vulnerable class. Complete the required classes and add meaningful tests.

3. Scenario

USP-Lite is a university application that stores and transfers student results.

The application must:

- verify user passwords;
- verify the integrity and authenticity of a result file;
- generate secure session tokens;
- encrypt confidential result data;
- reject modified or unauthenticated messages.

The file `data/grades.json` contains:

```
{
  "studentId": "S2026001",
  "course": "Software Security",
  "grade": 15.5,
  "semester": "2026-2027"
}
```

4. Task 1 — Analyze insecure cryptographic code

Read `VulnerableCrypto.java`.

Identify at least six weaknesses, explain their consequences, and propose corrections.

Complete this table in your report:

Code element	Observation	Security impact	Recommended correction
Password handling			
Token generation			
Key management			
AES mode			
Message authentication			
Error handling			

Answer:

1. Which mechanism should store a password?
2. Which mechanism should generate a session token?
3. Which mechanism authenticates a message using a shared secret?
4. Which AES configuration provides confidentiality and integrity?
5. Why can a secure algorithm still be used in insecure software?

5. Task 2 — Password hashing

Complete `PasswordService.java`.

Required operations:

```
String hash(String password);
boolean verify(String password, String storedHash);
boolean needsRehash(String storedHash);
```

Requirements:

- do not use MD5, SHA-1, or raw SHA-256 for password storage;
- use the approved password-hashing implementation supplied with the project;
- use a different random salt for every password;
- store the information required for verification;
- reject malformed hashes safely.

Perform these experiments:

1. Hash the same password twice.
2. Compare the two hashes.
3. Verify the correct password.
4. Verify a wrong password.
5. Verify a malformed hash.
6. Test `needsRehash`.

Add JUnit tests for all cases.

Answer:

1. Why should the application never decrypt a stored password?
2. Why should two hashes of the same password be different?
3. Does the salt need to be secret?
4. What is the difference between a salt and an encryption key?
5. Why should password parameters be upgradeable?

6. Task 3 — File integrity with SHA-256

Write a Java program that reads `data/grades.json`, calculates SHA-256, displays the digest, and saves it separately.

Then:

1. Change the grade from `15.5` to `19.0`.
2. Calculate the digest again.
3. Compare both digests.
4. Modify both the file and the digest file.
5. Explain why a public digest alone cannot authenticate the sender.

Answer:

1. Does SHA-256 prove sender identity?
2. What can an attacker do if both data and digest can be modified?
3. When is a public digest still useful?

7. Task 4 — Message authentication with HMAC

Complete `HmacService.java` using `HmacSHA256`.

Requirements:

- use `javax.crypto.Mac`;
- generate a random key;
- compare tags with `MessageDigest.isEqual()`;
- reject modified data;
- reject a modified tag;
- reject an incorrect key;
- reject a truncated tag.

Perform these experiments:

1. Calculate an HMAC for `grades.json`.
2. Verify the original file.
3. Modify the file.
4. Verify the modified file.
5. Modify the tag.
6. Use an incorrect key.
7. Use a truncated tag.

Answer:

1. Why cannot an attacker recompute a valid HMAC without the key?
2. Why should a timing-resistant comparison be used?
3. When is a digital signature preferable to HMAC?
4. What is the risk of hardcoding the HMAC key?

Add JUnit tests for every success and rejection case.

8. Task 5 — Secure token generation

Complete `TokenService.java` using `SecureRandom`.

Requirements:

- use `java.security.SecureRandom`;
- generate at least 32 random bytes;
- use URL-safe Base64 without padding;
- do not use `java.util.Random`;
- do not use timestamps as secrets;
- do not print tokens in logs.

Perform these experiments:

1. Generate 1,000 tokens.
2. Check their length.
3. Check their character set.
4. Search for collisions.
5. Compare `Random` and `SecureRandom`.
6. Explain why a predictable seed is dangerous.

Answer:

1. Why is `java.util.Random` unsuitable for session tokens?
2. What is the approximate entropy of 32 random bytes?
3. Why is a timestamp not a secret?
4. How should a server invalidate a session token?

9. Task 6 — Authenticated encryption with AES-GCM

Complete `AesGcmService.java` using AES-256-GCM.

The `EncryptedMessage` record contains:

```
version | iv | ciphertext | aad
```

Requirements:

- use `AES/GCM/NoPadding`;

- use a 32-byte AES key;
- generate a fresh 12-byte IV for every encryption;
- use a 128-bit authentication tag;
- support AAD;
- reject modified ciphertext;
- reject modified AAD;
- reject an incorrect key;
- reject malformed input.

Perform these experiments:

1. Encrypt `grades.json`.
2. Decrypt it with the correct key.
3. Modify one byte of the ciphertext.
4. Modify the IV.
5. Modify the AAD.
6. Use a different key.
7. Truncate the ciphertext.
8. Record the failure behavior.

Answer:

1. What properties does AES-GCM provide?
2. What is the role of the IV?
3. What is the role of the authentication tag?
4. What is the role of AAD?
5. Why must the IV be different for every encryption?
6. Why must the AES key not be written in source code?
7. What should the application do when authentication fails?

Vulnerable experiment

Create a separate experimental implementation using a fixed IV. Encrypt the same plaintext twice and encrypt two different plaintexts. Record the results and explain why the implementation must not be used in a real system.

Do not use the vulnerable version in the corrected implementation.